

FastAPI Cheatsheet

From Zero to Production



Azeem Teli

AZEEM TELI

FastAPI Cheatsheet: From Zero to Production

Quick References From Beginner To Advanced

Copyright © 2025 by Azeem Teli

All rights reserved. No part of this publication may be reproduced, stored or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning, or otherwise without written permission from the publisher. It is illegal to copy this book, post it to a website, or distribute it by any other means without permission.

Azeem Teli asserts the moral right to be identified as the author of this work.

Azeem Teli has no responsibility for the persistence or accuracy of URLs for external or third-party Internet Websites referred to in this publication and does not guarantee that any content on such Websites is, or will remain, accurate or appropriate.

Designations used by companies to distinguish their products are often claimed as trademarks. All brand names and product names used in this book and on its cover are trade names, service marks, trademarks and registered trademarks of their respective owners. The publishers and the book are not associated with any product or vendor mentioned in this book. None of the companies referenced within the book have endorsed the book.

First edition

This book was professionally typeset on Reedsy.

Find out more at reedsy.com

Contents

I Getting Started

1	What is FastAPI	3
2	Installation & Setup	5
3	First FastAPI App	7
4	Running the Server with Uvicorn	9

II Core Basics

5	Path Parameters	13
6	Query Parameters	15
7	Request Body (Pydantic Models)	17
8	Response Models & Status Codes	20
9	Handling Form Data & Files Upload	23
10	Error Handling & Custom Exceptions	25

III Intermediate Concepts

11	Dependency Injection Basics	31
12	Path & Query Validation with Pydantic	33
13	Request Headers & Cookies	35
14	Middleware in FastAPI	37
15	Background Tasks	39
16	Logging in FastAPI	41

IV Authentication & Security

17	OAuth2 with Password Flow	45
18	JWT Authentication	47
19	Role-Based Access Control (RBAC)	49
20	CORS (Cross-Origin Resource Sharing)	51

V Database Integration

21	Connecting to Databases	55
22	SQLAlchemy with FastAPI	57
23	Alembic Migrations	60
24	Async Database Queries	62
25	CRUD Operations	65
26	Supabase Integration	68

VI APIs & Advanced Features

27	FastAPI with WebSockets	73
28	FastAPI + GraphQL	75
29	File Streaming & Large File Uploads	78
30	Pagination in APIs	80
31	Rate Limiting	82

VII Testing & Performance

32	Writing Unit Tests with pytest	87
33	Testing Routes with TestClient	89
34	Async Tests	91
35	Caching with Redis	93
36	Performance Tuning & Benchmarks	95

VIII Deployment & Production

37	Environment Variables & Config Management	101
38	Dockerizing FastAPI Apps	103
39	Deploying on Cloud	105
40	Using Gunicorn & Uvicorn Workers	108
41	CI/CD Pipelines for FastAPI	110

IX Tools & Ecosystem

42	FastAPI with Frontend	115
43	FastAPI with Celery (Task Queues)	117
44	API Documentation (Swagger & ReDoc)	120
45	OpenAPI Schema Customization	122
46	Monitoring & Observability	125

X Master Tips & Patterns

47	Best Practices for Project Structure	131
48	Dependency Injection – Advanced Patterns	133
49	Event Handling (startup/shutdown events)	136
50	Versioning APIs	138
51	Scaling FastAPI Apps (Microservices & gRPC)	140

	<i>About the Author</i>	143
--	-------------------------	-----

	<i>Also by Azeem Teli</i>	145
--	---------------------------	-----

I

Getting Started

1

What is FastAPI

FastAPI is a modern, fast (high-performance) web framework for building APIs with **Python 3.7+** based on **standard type hints**.

- ◆ Created by **Sebastián Ramírez**.
- ◆ Built on top of **Starlette** (for web handling) and **Pydantic** (for data validation).

⚡ Key Features

- **Speed:** Comparable to Node.js & Go (thanks to ASGI + async).
- **Type Hints:** Automatic request validation, autocompletion, fewer bugs.
- **Docs Built-in:** Interactive API docs (Swagger UI, ReDoc).
- **Asynchronous Support:** Native async/await.
- **Data Handling:** Strong with Pydantic (parsing & validation).

✓ Why FastAPI over Flask/Django?

- **Flask:** Lightweight but lacks built-in validation, async, and auto-docs. You add them with extensions.
- **Django:** Full web framework (ORM, templates, admin) — better for monolithic apps, but heavy for APIs.
- **FastAPI:** Best for **APIs & microservices**, where performance, validation,

and developer productivity matter.

When to Choose FastAPI

- Building REST/GraphQL APIs.
- Microservices, backend for SPAs or mobile apps.
- High-performance async applications (chat apps, real-time APIs).

2

Installation & Setup

Install FastAPI

```
pip install fastapi
```

Install Uvicorn (server)

```
pip install "uvicorn[standard]"
```

Create main.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
```

```
def read_root():  
    return {"message": "Hello FastAPI"}
```

Run the Server

```
uvicorn main:app --reload
```

- main → filename (main.py)
- app → FastAPI instance
- —reload → auto-restart on code change

Default Local URL

- Docs (Swagger UI): <http://127.0.0.1:8000/docs>
- Alternative Docs (ReDoc): <http://127.0.0.1:8000/redoc>

3

First FastAPI App

 Create main.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def hello():
    return {"message": "Hello, FastAPI!"}
```

▶  Run App

```
uvicorn main:app --reload
```

Open in Browser

- API Root: <http://127.0.0.1:8000/> → {“message”: “Hello, FastAPI!”}
- Swagger Docs: <http://127.0.0.1:8000/docs>
- ReDoc: <http://127.0.0.1:8000/redoc>

4

Running the Server with Uvicorn

▶ ? Basic Run

```
uvicorn main:app
```

- main → file name (main.py)
- app → FastAPI instance

↻ Auto Reload (for dev)

```
uvicorn main:app --reload
```

🌐 Change Host & Port

```
uvicorn main:app --host 0.0.0.0 --port 8080
```

- `—host 0.0.0.0` → accessible from network
- `—port 8080` → custom port

Production Example

```
uvicorn main:app --workers 4
```

- `—workers` = number of processes

II

Core Basics

5

Path Parameters

Basic Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

 Visiting <http://127.0.0.1:8000/items/10> →

```
{"item_id": 10}
```

Multiple Parameters

```
@app.get("/users/{user_id}/orders/{order_id}")
def get_order(user_id: int, order_id: int):
    return {"user": user_id, "order": order_id}
```

Path Type Conversion

- int → /items/10 ✓ /items/abc ✗
- str → /items/abc ✓
- float → /items/3.14 ✓

Predefined Paths vs Dynamic

```
@app.get("/users/me")
def get_me():
    return {"user": "current user"}
```

✓ Always checked before dynamic /users/{user_id}.

6

Query Parameters

Basic Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/")
def read_item(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

🔑 URL → <http://127.0.0.1:8000/items/?skip=5&limit=20>

```
{"skip": 5, "limit": 20}
```

Optional Query Parameter

```
from typing import Optional

@app.get("/search/")
def search(q: Optional[str] = None):
    return {"query": q}
```

🔑 `/search/` → `{"query": null}`

🔑 `/search/?q=fastapi` → `{"query": "fastapi"}`

⚡ Multiple Query Params

```
@app.get("/filter/")
def filter_items(category: str, price: int):
    return {"category": category, "price": price}
```

🔑 `/filter/?category=books&price=100` →

```
{"category": "books", "price": 100}
```

Default Values

```
@app.get("/users/")
def get_users(active: bool = True):
    return {"active": active}
```

🔑 `/users/` → `{"active": true}`

7

Request Body (Pydantic Models)

Define Model

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True
```

Send Request Body

```
@app.post("/items/")
def create_item(item: Item):
    return item
```

 POST → /items/ with JSON body:

```
{
  "name": "Laptop",
  "price": 1200.50
}
```

✓ Response →

```
{
  "name": "Laptop",
  "price": 1200.5,
  "in_stock": true
}
```

🕒 Automatic Validation

- Missing field → ✗ error
- Wrong type → ✗ error

```
{
  "name": "Book",
  "price": "abc"
}
```

➡ Returns 422 Unprocessable Entity

⚡ Nested Models

```
class User(BaseModel):
    username: str
    email: str
```

REQUEST BODY (PYDANTIC MODELS)

```
class Order(BaseModel):
    item: Item
    buyer: User

@app.post("/orders/")
def create_order(order: Order):
    return order
```

Default + Optional Fields

```
from typing import Optional

class Product(BaseModel):
    name: str
    description: Optional[str] = None
    price: float = 0
```

8

Response Models & Status Codes

Basic Response Model

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.get("/item/", response_model=Item)
def get_item():
    return {"name": "Phone", "price": 500}
```

✓ Response auto-validated against Item model.

List Response

```
@app.get("/items/", response_model=list[Item])
def get_items():
    return [{"name": "Book", "price": 20}, {"name": "Laptop",
        "price": 1000}]
```

⚡ Exclude Fields with response_model_exclude

```
class User(BaseModel):
    username: str
    password: str

@app.get("/user/", response_model=User,
response_model_exclude={"password"})
def get_user():
    return {"username": "azeem", "password": "secret"}
```

Custom Status Codes

```
from fastapi import status

@app.post("/create/", status_code=status.HTTP_201_CREATED)
def create_item():
    return {"msg": "Created!"}
```

Raise HTTP Exceptions

```
from fastapi import HTTPException
```

```
@app.get("/error/")
def error():
    raise HTTPException(status_code=404, detail="Item not found")
```

Handling Form Data & Files Upload

Form Data

```
from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/login/")
def login(username: str = Form(...), password: str = Form(...)):
    return {"username": username}
```

 Send as application/x-www-form-urlencoded

✓ Request Body Example:

```
username=azeem&password=1234
```

File Upload (Single File)

```
from fastapi import File, UploadFile

@app.post("/upload/")
def upload_file(file: UploadFile = File(...)):
    return {"filename": file.filename, "type": file.content_type}
```

🔑 Upload via Form-Data (multipart/form-data)

🕒 Multiple File Uploads

```
@app.post("/upload-multiple/")
def upload_files(files: list[UploadFile] = File(...)):
    return {"filenames": [f.filename for f in files]}
```

⚡ Read File Contents

```
@app.post("/upload/read/")
async def read_file(file: UploadFile = File(...)):
    content = await file.read()
    return {"size": len(content)}
```

Notes

- Form(...) → for text input in forms
- File(...) → for binary uploads
- UploadFile → gives .filename, .content_type, .read()

10

Error Handling & Custom Exceptions

Raise Built-in HTTP Exception

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id == 0:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item_id": item_id}
```

 Returns →

```
{"detail": "Item not found"}
```

Custom Exception Class

```
class MyException(Exception):
    def __init__(self, name: str):
        self.name = name
```

? Exception Handler

```
from fastapi import Request
from fastapi.responses import JSONResponse

@app.exception_handler(MyException)
def my_exception_handler(request: Request, exc: MyException):
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name} did something wrong."},
    )
```

🔑 Usage:

```
@app.get("/test/")
def test():
    raise MyException("Azeem")
```

✓ Response →

```
{"message": "Oops! Azeem did something wrong."}
```

⚡ Override Default Exception

```
from fastapi.exceptions import RequestValidationError

@app.exception_handler(RequestValidationError)
def validation_exception_handler(request, exc):
    return JSONResponse(
        status_code=400,
        content={"errors": exc.errors()},
    )
```

Tips

- Use HTTPException for common API errors.
- Use custom classes + handlers for app-specific errors.
- Return consistent JSON error structure.

III

Intermediate Concepts

11

Dependency Injection Basics

Simple Dependency

```
from fastapi import FastAPI, Depends

app = FastAPI()

def common_query(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}

@app.get("/items/")
def read_items(pagination: dict = Depends(common_query)):
    return pagination
```

 URL → /items/?skip=5&limit=20

✓ Response → {"skip": 5, "limit": 20}

Class as Dependency

```

class CommonParams:
    def __init__(self, q: str = None, page: int = 1):
        self.q = q
        self.page = page

@app.get("/search/")
def search(params: CommonParams = Depends()):
    return {"query": params.q, "page": params.page}

```

⚡ Shared Dependencies Across Routes

```

async def verify_token(token: str = None):
    if token != "abc123":
        raise Exception("Invalid Token")

@app.get("/protected/", dependencies=[Depends(verify_token)])
def protected():
    return {"msg": "Access Granted"}

```

Key Points

- Depends() lets you **inject reusable logic**.
- Works with **functions, classes, async functions**.
- Great for **auth, pagination, DB connections, validation**.
- Keeps code **DRY & clean**.

12

Path & Query Validation with Pydantic

Path Parameter Validation

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(
    item_id: int = Path(..., title="The ID of the item", ge=1,
                        le=1000)
):
    return {"item_id": item_id}
```

- ✓ ge=1 → greater or equal to 1
- ✓ le=1000 → less or equal to 1000

Query Parameter Validation

```

from fastapi import Query
from typing import Optional

@app.get("/search/")
def search(
    q: Optional[str] = Query(None, min_length=3, max_length=50,
    regex="^fast.*")
):
    return {"query": q}

```

- ✓ `min_length` / `max_length` → string length limits
- ✓ `regex` → validate pattern

⚡ Multiple Query Params with Validation

```

@app.get("/filter/")
def filter_items(
    price: float = Query(..., gt=0, lt=10000),
    category: str = Query(..., min_length=2)
):
    return {"price": price, "category": category}

```

- ✓ `gt` → greater than
- ✓ `lt` → less than

Key Points

- Use `Path()` for path parameters, `Query()` for query parameters.
- Validation is **automatic**.
- Errors return **422 Unprocessable Entity** with details.
- Works with **default values** and **optional fields**.

13

Request Headers & Cookies

Access Request Headers

```
from fastapi import FastAPI, Header
from typing import Optional

app = FastAPI()

@app.get("/header/")
def read_header(user_agent: Optional[str] = Header(None)):
    return {"User-Agent": user_agent}
```

 Automatically reads header User-Agent from request.

Multiple Headers

```
@app.get("/headers/")
def read_headers(
    x_token: str = Header(...),
    x_key: Optional[str] = Header(None)
):
```

```
return {"X-Token": x_token, "X-Key": x_key}
```

✓ Required headers → ...

✓ Optional headers → Optional[...]

⚡ Access Cookies

```
from fastapi import Cookie

@app.get("/cookie/")
def read_cookie(ads_id: Optional[str] = Cookie(None)):
    return {"ads_id": ads_id}
```

🔑 Read cookie ads_id from request

Key Points

- Header() → Read request headers
- Cookie() → Read cookies
- Use Optional[...] for optional values
- FastAPI **auto-validates types** (int, str, bool)

14

Middleware in FastAPI

Basic Middleware Example

```
from fastapi import FastAPI
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import Response

app = FastAPI()

@app.middleware("http")
async def simple_middleware(request: Request, call_next):
    print(f"Request URL: {request.url}")
    response = await call_next(request)
    response.headers["X-Custom-Header"] = "FastAPI"
    return response
```

 Runs **before and after every request**

Using BaseHTTPMiddleware

```
class CustomMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        print("Before Request")
        response = await call_next(request)
        print("After Request")
        return response

app.add_middleware(CustomMiddleware)
```

⚡ Use Cases

- Logging requests/responses
- Authentication checks
- Adding custom headers
- Measuring request time
- Rate limiting

Key Points

- Middleware **wraps every request**
- Can be **function-based** (`@app.middleware`) or **class-based** (`BaseHTTPMiddleware`)
- Always call `await call_next(request)` to continue the request chain

15

Background Tasks

Basic Example

```
from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(message + "\n")

@app.post("/send-notification/")
def send_notification(background_tasks: BackgroundTasks, msg: str):
    background_tasks.add_task(write_log, msg)
    return {"message": "Notification sent in background"}
```

✓ write_log runs **after response is returned**

Multiple Background Tasks

```
def task1():
    print("Task 1 done")

def task2():
    print("Task 2 done")

@app.post("/multi-task/")
def multi_task(background_tasks: BackgroundTasks):
    background_tasks.add_task(task1)
    background_tasks.add_task(task2)
    return {"status": "Tasks scheduled"}
```

⚡ Async Background Task

```
import asyncio

async def async_task():
    await asyncio.sleep(2)
    print("Async task completed")
```

✓ Can add async functions too: `background_tasks.add_task(async_task)`

Key Points

- Background tasks **don't block the main request**
- Perfect for **email sending, logging, file processing**
- Use `BackgroundTasks` parameter in endpoint
- Supports multiple tasks and async functions

16

Logging in FastAPI

Basic Logging

```
import logging
from fastapi import FastAPI

app = FastAPI()

logging.basicConfig(level=logging.INFO)

@app.get("/info/")
def get_info():
    logging.info("Info endpoint called")
    return {"message": "Check logs"}
```

✓ Logs appear in console

Custom Logger

```

logger = logging.getLogger("my_logger")
logger.setLevel(logging.DEBUG)

@app.get("/debug/")
def debug_log():
    logger.debug("Debug message")
    logger.warning("Warning message")
    return {"status": "Check logs"}

```

⚡ Logging Requests with Middleware

```

from starlette.requests import Request

@app.middleware("http")
async def log_requests(request: Request, call_next):
    logger.info(f"Request URL: {request.url}")
    response = await call_next(request)
    logger.info(f"Response status: {response.status_code}")
    return response

```

Key Points

- `logging.basicConfig(level=...)` sets default log level
- Levels: `DEBUG < INFO < WARNING < ERROR < CRITICAL`
- Use **custom loggers** for different modules
- Middleware logging helps track requests/responses

IV

Authentication & Security

17

OAuth2 with Password Flow

Setup Dependencies

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
```

```
app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
```

Token Endpoint

```
@app.post("/token")
def login(form_data: OAuth2PasswordRequestForm = Depends()):
    if form_data.username != "azeem" or form_data.password !=
    "1234":
        raise
        HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid credentials")
```

```
return {"access_token": form_data.username + "_token",  
        "token_type": "bearer"}
```

🔑 OAuth2PasswordRequestForm reads username & password from form data

⚡ Protect Routes

```
@app.get("/users/me")  
def read_users_me(token: str = Depends(oauth2_scheme)):  
    return {"token": token}
```

✓ Requires valid token from /token endpoint

Key Points

- OAuth2PasswordBearer(tokenUrl="token") → defines where clients get tokens
- Use **Depends(oauth2_scheme)** to protect endpoints
- Tokens are usually **JWT or opaque tokens**
- FastAPI auto-generates Swagger UI for token auth

18

JWT Authentication

Install Dependency

```
pip install pyjwt passlib[bcrypt]
```

Create & Verify Token

```
from datetime import datetime, timedelta
import jwt

SECRET_KEY = "mysecret"

def create_token(data: dict, expires_delta: int = 15):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=expires_delta)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY,
                             algorithm="HS256")
    return encoded_jwt
```

```
def verify_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY,
                               algorithms=["HS256"])
        return payload
    except jwt.ExpiredSignatureError:
        return None
```

⚡ FastAPI Dependency

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def get_current_user(token: str = Depends(oauth2_scheme)):
    payload = verify_token(token)
    if not payload:
        raise
        HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                      detail="Invalid or expired token")
    return payload
```

Protect Routes

```
@app.get("/users/me")
def read_users_me(current_user: dict = Depends(get_current_user)):
    return current_user
```

✓ Only valid JWT tokens can access /users/me

Role-Based Access Control (RBAC)

Define Users & Roles

```
from fastapi import FastAPI, Depends, HTTPException, status

app = FastAPI()

fake_users_db = {
    "azeem": {"username": "azeem", "role": "admin"},
    "ali": {"username": "ali", "role": "user"}
}

def get_current_user(username: str):
    user = fake_users_db.get(username)
    if not user:
        raise
        HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
            detail="User not found")
    return user
```

🕒 Role Check Dependency

```
def require_role(role: str):
    def role_checker(user: dict = Depends(get_current_user)):
        if user["role"] != role:
            raise
            HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                          detail="Operation not permitted")
        return user
    return role_checker
```

⚡ Protect Routes by Role

```
@app.get("/admin/")
def admin_dashboard(user: dict = Depends(require_role("admin"))):
    return {"msg": f"Welcome {user['username']}!"}

@app.get("/user/")
def user_dashboard(user: dict = Depends(require_role("user"))):
    return {"msg": f"Welcome {user['username']}!"}
```

✓ Only users with proper roles can access specific endpoints

Key Points

- Create a **dependency** that checks user role
- Combine with JWT / OAuth2 for secure APIs
- Returns **403 Forbidden** if role doesn't match
- Works with multiple roles (extend `require_role` to a list)

CORS (Cross-Origin Resource Sharing)

Install Dependency

```
pip install fastapi[all]
```

(or make sure starlette is installed, usually comes with FastAPI)

Enable CORS

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost",
    "http://localhost:3000",
    "https://myfrontend.com"
]
```

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

⚡ Key Parameters

- `allow__origins` → List of allowed domains
- `allow__credentials` → Allow cookies, auth headers
- `allow__methods` → HTTP methods (GET, POST...)
- `allow__headers` → Headers allowed in requests

Key Points

- Necessary when frontend and backend are **on different domains/ports**
- Using “*” allows all origins (not recommended for production)
- Works with **JS fetch, Axios, or any browser requests**
- Always configure carefully for **security**

V

Database Integration

21

Connecting to Databases

Install Dependencies

```
pip install sqlalchemy psycopg2-binary
```

- sqlalchemy → ORM
- psycopg2-binary → PostgreSQL driver
- For SQLite, no extra driver needed

SQLite Example

```
from sqlalchemy import create_engine

DATABASE_URL = "sqlite:///./test.db"
engine = create_engine(DATABASE_URL,
    connect_args={"check_same_thread": False})
```

✓ Creates test.db in current folder

⚡ PostgreSQL Example

```
DATABASE_URL =  
"postgresql://user:password@localhost:5432/mydatabase"  
engine = create_engine(DATABASE_URL)
```

✓ Replace user, password, localhost, 5432, mydatabase with your credentials

Key Points

- Use **SQLAlchemy** to interact with DB (ORM or raw SQL)
- SQLite → great for dev/testing, PostgreSQL → production
- engine → main connection object
- Consider **async database libraries** (databases, asyncpg) for high-performance apps

SQLAlchemy with FastAPI

Install Dependencies

```
pip install sqlalchemy psycopg2-binary
```

- Use sqlite or postgresql URL as needed

Setup Database & Session

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///./test.db"

engine = create_engine(DATABASE_URL,
    connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
    bind=engine)
Base = declarative_base()
```

⚡ Create Models

```
from sqlalchemy import Column, Integer, String

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    email = Column(String, unique=True, index=True)
```

📄 Dependency for DB Session

```
from fastapi import Depends

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

CRUD Example

```
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session

app = FastAPI()

@app.post("/users/")
```

```
def create_user(name: str, email: str, db: Session =  
Depends(get_db)):  
    user = User(name=name, email=email)  
    db.add(user)  
    db.commit()  
    db.refresh(user)  
    return user
```

Key Points

- `Base.metadata.create_all(engine)` → create tables
- `SessionLocal()` → manage DB session
- Use `Depends(get_db)` in endpoints
- Works with **SQLite for dev, PostgreSQL/MySQL for production**

23

Alembic Migrations

Install Alembic

```
pip install alembic
```

Initialize Alembic

```
alembic init alembic
```

- Creates alembic folder + alembic.ini

Configure alembic.ini

```
sqlalchemy.url = sqlite:///./test.db
```

- Replace with PostgreSQL if needed:
- postgresql://user:password@localhost/dbname

Create Migration

```
alembic revision --autogenerate -m "create users table"
```

- Auto-generates migration based on SQLAlchemy models

Apply Migration

```
alembic upgrade head
```

- Applies latest migration to database

◀ Downgrade Migration

```
alembic downgrade -1
```

- Rolls back last migration

Key Points

- Use **autogenerate** for automatic migration scripts
- Keep **models and migrations in sync**
- Use **head** to apply latest migration
- Essential for **production database schema changes**

Async Database Queries

Install Dependencies

```
pip install databases sqlalchemy aiosqlite asyncpg
```

- databases → async DB access
- aiosqlite → async SQLite driver
- asyncpg → async PostgreSQL driver

Setup Async Database

```
from databases import Database

DATABASE_URL = "sqlite+aiosqlite:///./test.db"
database = Database(DATABASE_URL)
```

⚡ Connect & Disconnect

```
from fastapi import FastAPI

app = FastAPI()

@app.on_event("startup")
async def startup():
    await database.connect()

@app.on_event("shutdown")
async def shutdown():
    await database.disconnect()
```

Async Query Example

```
@app.get("/users/")
async def get_users():
    query = "SELECT * FROM users"
    return await database.fetch_all(query)
```

Async CRUD Example

```
@app.post("/users/")
async def create_user(name: str, email: str):
    query = "INSERT INTO users(name, email) VALUES (:name, :email)"
    user_id = await database.execute(query, values={"name": name,
    "email": email})
    return {"id": user_id, "name": name, "email": email}
```

Key Points

- Use **async/await** for database operations
- databases library works with **SQLAlchemy models or raw SQL**
- Improves **performance in high-concurrency apps**
- Always connect/disconnect DB in **startup/shutdown events**

CRUD Operations

Create (POST)

```
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session
from models import User, get_db

app = FastAPI()

@app.post("/users/")
def create_user(name: str, email: str, db: Session =
Depends(get_db)):
    user = User(name=name, email=email)
    db.add(user)
    db.commit()
    db.refresh(user)
    return user
```

Read (GET)

```
@app.get("/users/{user_id}")
def read_user(user_id: int, db: Session = Depends(get_db)):
    return db.query(User).filter(User.id == user_id).first()
```

✓ List all users

```
@app.get("/users/")
def read_users(db: Session = Depends(get_db)):
    return db.query(User).all()
```

⚡ Update (PUT / PATCH)

```
@app.put("/users/{user_id}")
def update_user(user_id: int, name: str, db: Session =
Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    user.name = name
    db.commit()
    db.refresh(user)
    return user
```

🗑 Delete (DELETE)

```
@app.delete("/users/{user_id}")
def delete_user(user_id: int, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    db.delete(user)
    db.commit()
    return {"msg": "User deleted"}
```

Key Points

- Use **POST** to create, **GET** to read, **PUT/PATCH** to update, **DELETE** to remove
- Always commit and refresh objects when using SQLAlchemy
- Use **Depends(get_db)** for database session injection
- Keep endpoints consistent with **REST conventions**

Supabase Integration

Install Dependencies

```
pip install supabase
```

Initialize Supabase Client

```
from supabase import create_client

url = "https://xyzcompany.supabase.co"
key = "your-anon-key"
supabase = create_client(url, key)
```

Fetch Data

```
@app.get("/users/")
def get_users():
    users = supabase.table("users").select("*").execute()
    return users.data
```

Insert Data

```
@app.post("/users/")
def create_user(name: str, email: str):
    res = supabase.table("users").insert({"name": name, "email":
    email}).execute()
    return res.data
```

Update Data

```
@app.put("/users/{user_id}")
def update_user(user_id: int, name: str):
    res = supabase.table("users").update({"name": name}).eq("id",
    user_id).execute()
    return res.data
```

Delete Data

```
@app.delete("/users/{user_id}")
def delete_user(user_id: int):
    res = supabase.table("users").delete().eq("id",
    user_id).execute()
    return res.data
```

Key Points

- Supabase is **PostgreSQL + Realtime + Auth**
- `.table("name")` → select table
- `.select("*")` → fetch all columns
- `.insert({...})`, `.update({...})`, `.delete()` → CRUD operations
- `.eq("column", value)` → filter rows

VI

APIs & Advanced Features

FastAPI with WebSockets

Basic WebSocket

```
from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    await websocket.send_text("Hello WebSocket!")
    await websocket.close()
```

🔑 Connect in browser or client → “ws://127.0.0.1:8000/ws”

🕒 Echo Server

```
@app.websocket("/ws/echo")
async def websocket_echo(websocket: WebSocket):
    await websocket.accept()
    while True:
```

```
data = await websocket.receive_text()
await websocket.send_text(f"Message received: {data}")
```

✔ Sends back whatever client sends

⚡ Broadcast to Multiple Clients

```
clients = []

@app.websocket("/ws/broadcast")
async def websocket_broadcast(websocket: WebSocket):
    await websocket.accept()
    clients.append(websocket)
    try:
        while True:
            msg = await websocket.receive_text()
            for client in clients:
                await client.send_text(msg)
    except:
        clients.remove(websocket)
```

Key Points

- Use WebSocket for **bi-directional communication**
- `await websocket.accept()` → must call first
- `receive_text()` / `send_text()` → exchange messages
- Can be used for **chat apps, live notifications, gaming**
- Keep track of **connected clients** for broadcasting

FastAPI + GraphQL

Install Dependencies

```
pip install graphene fastapi-graphql
```

Basic GraphQL Setup

```
import graphene
from fastapi import FastAPI
from starlette.graphql import GraphQLApp

app = FastAPI()

class Query(graphene.ObjectType):
    hello = graphene.String(name=graphene.String(default_value="world"))

    def resolve_hello(root, info, name):
        return f"Hello {name}!"
```

```
app.add_route("/graphql",
GraphQLApp(schema=graphene.Schema(query=Query)))
```

🔑 Access GraphQL playground → <http://127.0.0.1:8000/graphql>

⚡ Example Query

```
{
  hello(name: "Azeem")
}
```

✓ Response:

```
{
  "data": {
    "hello": "Hello Azeem!"
  }
}
```

📄 Adding Mutations

```
class CreateUser(graphene.Mutation):
    class Arguments:
        name = graphene.String()

        ok = graphene.Boolean()
        message = graphene.String()

    def mutate(root, info, name):
        return CreateUser(ok=True, message=f"User {name} created")

class Mutation(graphene.ObjectType):
```

```
create_user = CreateUser.Field()

app.add_route("/graphql",
GraphQLApp(schema=graphene.Schema(query=Query, mutation=Mutation)))
```

✓ Mutation Query Example:

```
mutation {
  createUser(name: "Ali") {
    ok
    message
  }
}
```

Key Points

- Use **Graphene** for schema & mutations
- FastAPI integrates with GraphQLApp
- Queries → resolve__<field>
- Mutations → class Mutation
- Playground → test queries interactively

File Streaming & Large File Uploads

Streaming Response

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

def iterfile():
    with open("large_file.txt", "rb") as f:
        yield from f

@app.get("/download/")
def download_file():
    return StreamingResponse(iterfile(),
                             media_type="application/octet-stream")
```

✓ Streams file in chunks instead of loading whole file into memory

Large File Uploads

```
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload/")
async def upload_file(file: UploadFile = File(...)):
    with open(f"uploads/{file.filename}", "wb") as f:
        while contents := await file.read(1024 * 1024):
            f.write(contents)
    return {"filename": file.filename}
```

✓ Reads file **in chunks (1 MB)** to avoid memory overload

⚡ Notes

- StreamingResponse → for **downloads, streaming content**
- Use await file.read(size) → handle **large uploads efficiently**
- Always close file handles to avoid memory leaks

Pagination in APIs

Simple Query Pagination

```
from fastapi import FastAPI, Query
from typing import List

app = FastAPI()

fake_items = [{"item_id": i} for i in range(1, 101)]

@app.get("/items/")
def get_items(skip: int = 0, limit: int = 10):
    return fake_items[skip : skip + limit]
```

 URL → /items/?skip=10&limit=20

Using Pydantic for Pagination Response

```
from pydantic import BaseModel
```



```
class PaginatedItems(BaseModel):
    total: int
    items: List[dict]

@app.get("/paginated-items/", response_model=PaginatedItems)
def paginated_items(skip: int = 0, limit: int = 10):
    return {"total": len(fake_items), "items":
        fake_items[skip:skip+limit]}
```

⚡ Key Points

- skip → number of items to skip
- limit → number of items to return
- Works for **arrays, database queries**
- Good practice: **return total count** for frontend pagination UI

Rate Limiting



Using slowapi

```
pip install slowapi
```

🕒 Setup FastAPI with Rate Limiter

```
from fastapi import FastAPI, Request
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

app = FastAPI()
limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded,
    _rate_limit_exceeded_handler)
```

⚡ Apply Rate Limit on Endpoints

```
@app.get("/limited/")
@limiter.limit("5/minute")
def limited_endpoint(request: Request):
    return {"msg": "You can access this endpoint 5 times per
        minute"}
```

✓ Returns **429 Too Many Requests** if exceeded

📄 Key Points

- Use `key_func` to identify client (IP by default)
- Rate limit format: "5/minute", "10/second", "100/hour"
- Protect sensitive or high-traffic endpoints
- Works with **async and sync endpoints**

VII

Testing & Performance

Writing Unit Tests with pytest

Install Dependencies

```
pip install pytest httpx
```

- httpx → async HTTP client for testing FastAPI

Basic Test for FastAPI Endpoint

```
from fastapi.testclient import TestClient
from main import app # your FastAPI app

client = TestClient(app)

def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello World"}
```

⚡ Test POST Endpoint with JSON

```
def test_create_user():
    response = client.post("/users/", json={"name": "Azeem",
    "email": "azeem@test.com"})
    assert response.status_code == 200
    assert response.json()["name"] == "Azeem"
```

📄 Async Tests

```
import pytest
from httpx import AsyncClient
from main import app

@pytest.mark.asyncio
async def test_async_endpoint():
    async with AsyncClient(app=app, base_url="http://test") as ac:
        response = await ac.get("/async-endpoint")
    assert response.status_code == 200
```

Key Points

- Use `TestClient` for sync endpoints, `httpx.AsyncClient` for async
- Test **status codes, response body, headers**
- Run tests → `pytest` in project directory
- Keep tests **small, independent, and repeatable**

Testing Routes with TestClient

Install Dependencies

```
pip install pytest
```

- TestClient comes with FastAPI (via starlette)

Basic Setup

```
from fastapi.testclient import TestClient
from main import app # your FastAPI app

client = TestClient(app)
```

Test GET Endpoint

```
def test_read_item():
    response = client.get("/items/1")
    assert response.status_code == 200
    assert response.json() == {"item_id": 1, "name": "Item 1"}
```

Test POST Endpoint

```
def test_create_item():
    response = client.post("/items/", json={"name": "NewItem",
    "price": 50})
    assert response.status_code == 200
    assert response.json()["name"] == "NewItem"
```

Test Query Parameters

```
def test_query_params():
    response = client.get("/items/?skip=5&limit=10")
    assert response.status_code == 200
    assert len(response.json()) == 10
```

Key Points

- Use `TestClient(app)` to simulate requests
- Test **GET, POST, PUT, DELETE** endpoints
- Include **query params, headers, cookies** as needed
- Works for **sync endpoints** (for async, use `httpx.AsyncClient`)

34

Async Tests

Install Dependencies

```
pip install pytest httpx
```

- httpx → async HTTP client for testing FastAPI

Basic Async Test

```
import pytest
from httpx import AsyncClient
from main import app # your FastAPI app

@pytest.mark.asyncio
async def test_async_endpoint():
    async with AsyncClient(app=app, base_url="http://test") as ac:
        response = await ac.get("/async-endpoint")
        assert response.status_code == 200
        assert response.json() == {"message": "Hello Async"}
```

⚡ Async POST Test

```
@pytest.mark.asyncio
async def test_create_user():
    async with AsyncClient(app=app, base_url="http://test") as ac:
        response = await ac.post("/users/", json={"name": "Azeem",
            "email": "azeem@test.com"})
    assert response.status_code == 200
    assert response.json()["name"] == "Azeem"
```

📝 Key Tips

- Use `@pytest.mark.asyncio` for async tests
- `AsyncClient` simulates async requests
- Works for **async endpoints only**
- Combine with fixtures to **setup test database**

Caching with Redis

Install Dependencies

```
pip install redis fastapi[all]
```

- redis → Redis client
- Make sure **Redis server is running** locally or remotely

Connect to Redis

```
import redis

r = redis.Redis(host='localhost', port=6379, db=0,
decode_responses=True)
```

Cache GET Response

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def get_item(item_id: int):
    cached = r.get(f"item:{item_id}")
    if cached:
        return {"item_id": item_id, "name": cached, "cached": True}

    # simulate DB fetch
    name = f"Item {item_id}"
    r.set(f"item:{item_id}", name, ex=60) # cache for 60 seconds
    return {"item_id": item_id, "name": name, "cached": False}
```

Key Points

- `r.set(key, value, ex=seconds)` → cache with expiration
- `r.get(key)` → fetch cached value
- Improves **performance & reduces DB load**
- Use for **frequently accessed endpoints**

Performance Tuning & Benchmarks

Use Uvicorn with Multiple Workers

```
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

- `--workers` → number of processes
- Each worker handles requests independently

Use Gunicorn + Uvicorn Workers (Production)

```
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker
```

- `-w` → number of workers
- `-k` → worker class (UvicornWorker for async)

⚡ Enable Keep-Alive & Timeout Settings

```
uvicorn main:app --timeout-keep-alive 60
```

- Adjust **timeout** for long-lived requests (WebSockets, uploads)

📄 Benchmark with hey or wrk

```
# hey
hey -n 1000 -c 50 http://127.0.0.1:8000/items/

# wrk
wrk -t2 -c100 -d30s http://127.0.0.1:8000/items/
```

- -n → total requests
- -c → concurrent clients
- -d → duration

⚡ Optimize FastAPI Code

- Use **async endpoints** for I/O-bound operations
- Avoid blocking calls
- Use **caching** (Redis, memory) for frequent data
- Limit **database queries**, use **pagination**

Key Points

- Run benchmarks before & after changes
- Use **Uvicorn workers** + **async** for best throughput
- Monitor CPU & memory usage during load tests

- Combine with caching, DB optimization for maximum performance

VIII

Deployment & Production

37

Environment Variables & Config Management

Install Dependencies

```
pip install python-dotenv pydantic
```

- python-dotenv → load .env files
- pydantic → structured settings management

Using .env File

```
# .env
DATABASE_URL=sqlite:///./test.db
SECRET_KEY=mysecret
DEBUG=True
```

⚡ Load Variables with python-dotenv

```
from dotenv import load_dotenv
import os

load_dotenv() # loads .env

DATABASE_URL = os.getenv("DATABASE_URL")
SECRET_KEY = os.getenv("SECRET_KEY")
DEBUG = os.getenv("DEBUG", "False") == "True"
```

📄 Using Pydantic Settings

```
from pydantic import BaseSettings

class Settings(BaseSettings):
    database_url: str
    secret_key: str
    debug: bool = False

    class Config:
        env_file = ".env"

settings = Settings()
```

✓ Access → `settings.database_url`, `settings.secret_key`

Key Points

- Keep **sensitive data out of code**
- `.env` for development, **environment variables** for production
- Pydantic Settings → type validation + default values
- Combine with **FastAPI startup events** for config initialization

Dockerizing FastAPI Apps

Create Dockerfile

```
# Use official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy app files
COPY . .

# Expose port
EXPOSE 8000

# Run Uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build Docker Image

```
docker build -t fastapi-app .
```

Run Docker Container

```
docker run -d --name myapp -p 8000:8000 fastapi-app
```

✓ Access app → <http://localhost:8000>

Docker Compose Example

```
version: '3.9'

services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=sqlite:///./test.db
```

- Use `docker-compose up --build` to start

Key Points

- `WORKDIR /app` → keeps container organized
- Expose **same port as Uvicorn**
- Environment variables can be passed via **Dockerfile or Compose**
- Combine with **PostgreSQL or Redis containers** for production

Deploying on Cloud



Heroku Deployment

1. Install Heroku CLI

```
heroku login
```

1. Create Procfile

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

1. Create Heroku app & push

```
git init  
heroku create my-fastapi-app
```

```
git add .
git commit -m "initial commit"
git push heroku main
```

✓ Access app → <https://my-fastapi-app.herokuapp.com>

🕒 AWS Deployment (Elastic Beanstalk / ECS)

- **Elastic Beanstalk:**
- Install AWS CLI & EB CLI
- Run `eb init` → configure app
- Deploy → `eb create` & `eb deploy`
- **ECS (Docker):**
- Push Docker image to **ECR**
- Create ECS service & run container

⚡ Render Deployment

1. Connect GitHub repo
2. Select **FastAPI** + Python environment
3. Set **build command** → `pip install -r requirements.txt`
4. Set **start command** → `uvicorn main:app --host 0.0.0.0 --port 10000`

📄 Key Points

- Use **environment variables** for secrets
- Configure **ports correctly** (Heroku `$PORT`, Render default 10000)
- Use **Docker** for consistent cloud deployments
- Monitor logs:

```
heroku logs --tail
```

DEPLOYING ON CLOUD

- Cloud options: Heroku (easy), AWS (scalable), Render (simple + free tier)

Using Gunicorn & Uvicorn Workers

Install Dependencies

```
pip install gunicorn uvicorn
```

Basic Command

```
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind  
0.0.0.0:8000
```

- `-w 4` → 4 workers
- `-k uvicorn.workers.UvicornWorker` → async worker
- `—bind` → host and port

⚡ Advanced Options

- Set **timeout** (seconds):

```
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --timeout
120
```

- Enable **access logs**:

```
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker
--access-logfile -
```

- Use **environment variables**:

```
export WORKERS=4
gunicorn main:app -w $WORKERS -k uvicorn.workers.UvicornWorker
```

📄 Key Points

- Gunicorn manages **multiple processes**
- Uvicorn handles **async requests**
- Combine for **high concurrency & production-ready app**
- Always monitor logs and performance during deployment

CI/CD Pipelines for FastAPI

CI/CD Pipelines for FastAPI

Basic CI/CD Flow

1. **Push Code** → GitHub/GitLab/Bitbucket
2. **Run Tests** → Unit & integration tests with pytest
3. **Build Docker Image** → Optional for containerized deployment
4. **Deploy** → Cloud (Heroku, AWS, Render, etc.)

GitHub Actions Example

.github/workflows/ci.yml

```
name: FastAPI CI

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
```

```

steps:
  - uses: actions/checkout@v3
  - name: Set up Python
    uses: actions/setup-python@v4
    with:
      python-version: 3.11
  - name: Install dependencies
    run: pip install -r requirements.txt
  - name: Run tests
    run: pytest

```

⚡ Docker Build & Deploy

```

- name: Build Docker image
  run: docker build -t my-fastapi-app .
- name: Push Docker image
  run: docker push mydockerhubuser/my-fastapi-app
- name: Deploy to Cloud
  run: ssh user@server "docker pull
    mydockerhubuser/my-fastapi-app && docker-compose up -d"

```

Key Points

- CI → runs tests on **every push or pull request**
- CD → automatically deploys **passing code to production**
- Combine **Docker, GitHub Actions, and Cloud services** for full automation
- Can integrate **linting, type checking, and security scans**

IX

Tools & Ecosystem

FastAPI with Frontend

React / Vue Integration

- **Frontend** → React or Vue app running on localhost:3000
- **Backend** → FastAPI on localhost:8000
- Use **CORS middleware** in FastAPI:

```
from fastapi.middleware.cors import CORSMiddleware

origins = ["http://localhost:3000"]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- React Example Fetch:

```
fetch("http://localhost:8000/items/")
  .then(res => res.json())
  .then(data => console.log(data))
```

🕒 HTMX Integration

- Use **HTML forms and hx-requests** without JS frameworks

```
<form hx-post="/submit" hx-target="#result">
  <input name="name">
  <button>Submit</button>
</form>
<div id="result"></div>
```

- FastAPI Endpoint:

```
from fastapi import Form

@app.post("/submit")
def submit(name: str = Form(...)):
    return {"message": f"Hello {name}"}
```

✓ Returns response directly to #result div

⚡ Key Points

- CORS is required for **React/Vue** cross-origin requests
- HTMX allows **SPA-like behavior** without heavy JS
- FastAPI can serve **JSON for frontend frameworks**
- Can combine **JWT or OAuth2** for secure APIs

FastAPI with Celery (Task Queues)

Install Dependencies

```
pip install celery[redis] fastapi
```

- redis → message broker
- Can also use RabbitMQ instead of Redis

Configure Celery

```
from celery import Celery

celery_app = Celery(
    "worker",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/0"
)
```

⚡ Create Background Task

```
@celery_app.task
def send_email(email: str):
    print(f"Sending email to {email}")
    return f"Email sent to {email}"
```

📄 Trigger Task from FastAPI

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/send-email/")
def trigger_email(email: str):
    send_email.delay(email)
    return {"msg": "Email task sent!"}
```

✓ `.delay()` → schedules task asynchronously

Run Worker

```
celery -A main.celery_app worker --loglevel=info
```

- `-A main.celery_app` → points to Celery instance

Key Points

- Use Celery for **long-running tasks** (emails, video processing)
- FastAPI triggers tasks asynchronously → non-blocking
- Redis or RabbitMQ acts as **broker & backend**
- Monitor tasks with **Flower**:

```
pip install flower  
celery -A main.celery_app flower
```

API Documentation (Swagger & ReDoc)

Default Documentation

- FastAPI automatically provides:
- **Swagger UI** → /docs
- **ReDoc** → /redoc

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id, "name": f"Item {item_id}"}
```

✓ Visit /docs or /redoc in browser

🕒 Customize Docs Title & Description


```
app = FastAPI(
    title="My FastAPI App",
    description="This is a FastAPI cheatsheet example",
    version="1.0.0"
)
```

⚡ Customize OpenAPI URL

```
app = FastAPI(
    openapi_url="/api/v1/openapi.json",
    docs_url="/api/v1/docs",
    redoc_url="/api/v1/redoc"
)
```

- Useful for **versioning** and hiding default docs



Add Metadata to Endpoints

```
@app.get("/items/{item_id}", summary="Get Item",
description="Fetch an item by its ID")
def read_item(item_id: int):
    return {"item_id": item_id}
```

- ✓ Shows **summary & description** in Swagger UI

Key Points

- /docs → Swagger UI (interactive, test endpoints)
- /redoc → ReDoc (clean, read-only docs)
- Can **customize title, version, and URLs**
- Use **endpoint metadata** to make docs clear for frontend & team

OpenAPI Schema Customization

Change OpenAPI Metadata

```
from fastapi import FastAPI

app = FastAPI(
    title="My API",
    description="Custom OpenAPI schema example",
    version="1.0.0",
    contact={
        "name": "Azeem Teli",
        "email": "azeem@example.com"
    },
    license_info={
        "name": "MIT",
        "url": "https://opensource.org/licenses/MIT"
    }
)
```

✓ Updates Swagger & ReDoc UI

🕒 Custom Operation IDs

```
@app.get("/items/{item_id}", operation_id="getItemById")
def read_item(item_id: int):
    return {"item_id": item_id}
```

- Useful for **client code generation**

⚡ Tags for Grouping Endpoints

```
@app.get("/users/", tags=["Users"])
def list_users():
    return [{"username": "azeem"}]

@app.get("/products/", tags=["Products"])
def list_products():
    return [{"name": "Laptop"}]
```

- ✓ Organizes endpoints in Swagger UI

📄 External Docs

```
app = FastAPI(
    openapi_tags=[
        {
            "name": "Users",
            "description": "Operations with users",
            "externalDocs": {"description": "User guide", "url":
                "https://example.com/users"}
        }
    ]
)
```

)

- Adds **links to documentation**

Key Points

- Customize **title, version, contact, license**
- Use **tags** to group endpoints
- Set **operation_id** for client code generation
- Add **externalDocs** for extra references
- Enhances **Swagger/ReDoc usability** for teams and clients

Monitoring & Observability

Use Logging

```
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

@app.get("/items/{item_id}")
def read_item(item_id: int):
    logger.info(f"Fetching item {item_id}")
    return {"item_id": item_id}
```

✓ Logs requests, errors, and important events

Prometheus Metrics

```
pip install prometheus-fastapi-instrumentator

from prometheus_fastapi_instrumentator import Instrumentator
```

```
instrumentator = Instrumentator()
instrumentator.instrument(app).expose(app)
```

- Exposes metrics at `/metrics`
- Integrates with **Grafana for dashboards**

⚡ Sentry for Error Tracking

```
pip install sentry-sdk

import sentry_sdk
from sentry_sdk.integrations.asgi import SentryAsgiMiddleware

sentry_sdk.init(dsn="your_sentry_dsn")
app.add_middleware(SentryAsgiMiddleware)
```

- ✓ Captures **exceptions, stack traces, and request context**

📄 Health Checks

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

- Useful for **load balancers** and uptime monitoring

Key Points

- Logging → track application behavior
- Prometheus → metrics & monitoring
- Sentry → error tracking in production
- Health endpoints → check app readiness & liveness
- Combine for **full observability in production**

X

Master Tips & Patterns

Best Practices for Project Structure

Recommended Structure

```

fastapi_app/ |_____
app/ | |_____
    main.py      # FastAPI app instance | |_____
    api/         # API route modules | |_____
        v1/ | | | |_____
            endpoints/ | | | |_____
                users.py | | | |_____
                items.py | | | |_____
                __init__.py | | | |_____
            __init__.py | |_____
        core/      # Config, settings, security | |_____
            config.py | |_____
            security.py | |_____
        models/    # SQLAlchemy/Pydantic models | |_____
            user.py | |_____
        db/        # Database session & utils | |_____
            session.py | |_____
  
```

```

services/      # Business logic / services | |
tasks/         # Background tasks / Celery  | |
__init__.py    |
tests/         # Unit & integration tests   | |
test_users.py  | |
__init__.py    |
requirements.txt |
Dockerfile     |
alembic/       # Alembic migrations         |
.env           |
README.md

```

🔑 Key Guidelines

- **Separate concerns** → routes, models, services, tasks
- **API versioning** → organize endpoints under `api/v1/`
- **Config & secrets** → store in `core/config.py` and `.env`
- **Background tasks** → Celery in `tasks/`
- **Database logic** → separate session & models
- **Tests** → mirror app structure, keep isolated

⚡ Benefits

- Scalable & maintainable for large apps
- Easy for multiple developers to collaborate
- Easier **CI/CD, deployment, and monitoring** integration
- Clean separation improves **readability and debugging**

Dependency Injection – Advanced Patterns

Basic Recap

```
from fastapi import Depends, FastAPI

app = FastAPI()

def get_query(q: str = None):
    return q

@app.get("/items/")
def read_items(query: str = Depends(get_query)):
    return {"query": query}
```

✓ `Depends()` injects functions into endpoints

Class-Based Dependencies

```
class CommonQueryParams:
```

```

def __init__(self, q: str = None, limit: int = 10, offset: int
= 0):
    self.q = q
    self.limit = limit
    self.offset = offset

@app.get("/items/")
def read_items(common: CommonQueryParams = Depends()):
    return {"query": common.q, "limit": common.limit, "offset":
common.offset}

```

- Encapsulates **common parameters**
- Reusable across multiple endpoints

⚡ Sub-Dependencies

```

def get_db():
    db = "Database Session"
    try:
        yield db
    finally:
        print("Close DB connection")

def get_user(db: str = Depends(get_db)):
    return f"User fetched using {db}"

@app.get("/user/")
def read_user(user: str = Depends(get_user)):
    return {"user": user}

```

✓ DI can **nest dependencies**

Conditional Dependencies

```
from fastapi import Header, HTTPException

def get_token_header(x_token: str = Header(...)):
    if x_token != "secret-token":
        raise HTTPException(status_code=400, detail="Invalid
        X-Token")
    return x_token

@app.get("/secure-data/")
def secure_data(token: str = Depends(get_token_header)):
    return {"token": token}
```

- Useful for **auth, feature toggles, API keys**

Key Patterns

- **Singleton services** → shared across endpoints
- **Scoped dependencies** → per-request injection
- **Sub-dependencies** → reuse DB sessions, auth checks, configs
- **Class-based dependencies** → clean & maintainable for complex apps
- **Exception handling in dependencies** → centralize validation

Event Handling (startup/shutdown events)

Startup Event

```
from fastapi import FastAPI

app = FastAPI()

@app.on_event("startup")
async def startup_event():
    print("App is starting...")
    # Example: connect to DB, initialize cache
```

✓ Runs **once when app starts**

Shutdown Event

```
@app.on_event("shutdown")
async def shutdown_event():
    print("App is shutting down...")
    # Example: close DB connection, cleanup resources
```

✓ Runs **once when app stops**

⚡ Using Async Tasks on Startup

```
@app.on_event("startup")
async def init_redis():
    app.state.redis = await connect_to_redis()
```

- Store clients in `app.state` for access in endpoints

📄 Dependency Cleanup with Events

```
@app.on_event("shutdown")
async def close_redis():
    await app.state.redis.close()
```

- Ensures proper resource **cleanup & no memory leaks**

Key Points

- startup → initialize services, DB, cache, scheduler
- shutdown → cleanup connections, close resources
- Can combine with **background tasks** and **Celery**
- Use `app.state` to share resources across app

Versioning APIs

Path-Based Versioning

```
from fastapi import FastAPI, APIRouter

app = FastAPI()

v1 = APIRouter(prefix="/api/v1")
v2 = APIRouter(prefix="/api/v2")

@v1.get("/items/")
def get_items_v1():
    return {"version": "v1", "items": ["item1", "item2"]}

@v2.get("/items/")
def get_items_v2():
    return {"version": "v2", "items": ["item1", "item2", "item3"]}

app.include_router(v1)
app.include_router(v2)
```

✓ Access via `/api/v1/items/` or `/api/v2/items/`

🕒 Header-Based Versioning

```
from fastapi import Header, HTTPException

@app.get("/items/")
def get_items(x_api_version: str = Header("v1")):
    if x_api_version == "v1":
        return {"version": "v1"}
    elif x_api_version == "v2":
        return {"version": "v2"}
    else:
        raise HTTPException(status_code=400, detail="Unsupported
        API version")
```

- Useful for **clients to specify version** without changing URL

⚡ Query Parameter Versioning

```
@app.get("/items/")
def get_items(version: str = "v1"):
    return {"version": version}
```

- Use `?version=v1` or `?version=v2`

📄 Best Practices

- **Path versioning** → most common & explicit
- **Header versioning** → hides version from URL
- **Maintain multiple routers** → clean separation
- **Deprecate old versions** → communicate with clients

Scaling FastAPI Apps (Microservices & gRPC)

Microservices Architecture

- Break monolithic FastAPI app into **smaller services**
- Each service handles **specific domain**
- Use **API Gateway** or **reverse proxy** (NGINX, Traefik)

Example structure:

```
services/ |_____  
users-service/ |_____  
orders-service/ |_____  
products-service/
```

- Each service runs independently → can scale **horizontally**

🕒 Inter-Service Communication

- **HTTP/REST** → simple & common
- **Message queues** → RabbitMQ / Kafka for async communication
- **Celery** → for background tasks across services

⚡ gRPC for Fast Communication

- gRPC → high-performance RPC framework
- FastAPI + gRPC → define **.proto file**, generate Python stubs

```
pip install grpcio grpcio-tools
```

Example .proto:

```
syntax = "proto3";

service UserService {
    rpc GetUser (UserRequest) returns (UserResponse);
}

message UserRequest {
    int32 id = 1;
}

message UserResponse {
    string name = 1;
    string email = 2;
}
```

- Use **gRPC client/server** for fast inter-service calls

Containerization & Orchestration

- Docker → containerize each microservice
- Kubernetes → orchestrate, scale, load-balance
- Environment variables → manage configs per service

Key Points

- Use **microservices** for horizontal scalability
- gRPC → low-latency, high-performance communication
- Async tasks & queues → decouple services
- Docker + Kubernetes → automated scaling & deployment
- Monitor & log **each service independently**



About the Author

Azeem Teli is a Python developer and writer known for making complex programming topics approachable and engaging. He is the creator behind the blog “[PyZilla](#)” and contributes to several technical publications, including “**Data Engineer Things**” and “[Better Programming](#).” His writing often adopts a conversational tone, likening coding to casual “chai-time gossip,” with the aim of demystifying concepts in Python, Django, and web development for his readers.

Azeem’s content is particularly popular among developers seeking to enhance their skills without feeling overwhelmed. He focuses on practical tutorials, clean code practices, and insights into tools like SQLAlchemy and Google’s Project IDX. His work has been featured in major publications, including “[Python in Plain English](#)” and “JavaScript in Plain English.”

In addition to his technical writing, Azeem is active on Medium, where he shares personal reflections on his journey as a developer. His articles often blend humor and storytelling, reflecting his belief that coding doesn’t have to be boring.

For those interested in learning Python or exploring web development, Azeem Teli’s work offers a refreshing and accessible perspective.

You can connect with me on:

 <https://azeemтели.com>

Subscribe to my newsletter:

 <https://bit.ly/azeemтели>

Also by Azeem Teli



Python Developer Cheatsheet

<https://bit.ly/python-developer-cheatsheet>

A Quick Reference Guide

